

PHASE 3

Project Design Phase

EduGenie - AI Powered Learning Assistant

Define the architecture, component responsibilities, data flow, interface structure, and integration points.

Prepared as a phase-wise academic submission document based on the supplied EduGenie-Ai project archive.

1. High-Level Architecture

Browser UI -> FastAPI application -> Task-specific service module -> Gemini API -> Structured response -> Browser rendering

Presentation	Task selection, text entry, result display, status messages	static/css/style.css and static/css/js/app.js
Web/API	Routing, request handling, response models, error translation	main.py
Task services	Question answering, explanation, quiz, summary, learning path	Imported module names in main.py
AI integration	Generate content using Google Gemini	requirements.txt + .env configuration
Configuration	Model selection, temperature, feature flags	.env

2. Backend Route Design

/	GET	Render web page	HTML
/health	GET	Service health check	JSON status
/qa	POST	Answer question	TextResponse
/explain	POST	Explain topic	TextResponse
/quiz	POST	Generate quiz	QuizResponse
/summarize	POST	Summarize input	TextResponse
/learn/recommendations	POST	Generate learning recommendations	LearningPathResponse

3. Frontend Interaction Design

- The selected task changes the label and placeholder of the input field.
- The learning task reveals an additional level selector.
- The Generate button sends an asynchronous fetch request and changes to a generating state.
- Text responses are HTML-escaped before insertion into the DOM.
- Quiz answers are rendered as buttons and compared with the supplied correct answer.
- A Copy action copies the visible result to the clipboard.

4. Error Handling Design

Backend task handlers wrap the task call in try/except and raise HTTP 502 when an exception occurs. The browser reads detail from a failed response and displays the message through a visible status box.

5. Data Model Design

The backend references GenerateRequest, TextResponse, QuizResponse, and LearningPathResponse. The JavaScript implies that GenerateRequest includes text and may include level for learning-path requests. TextResponse contains result. QuizResponse is expected to contain a result object with questions.

6. Security Design Considerations

- Keep GEMINI_API_KEY in environment configuration and out of source files and submission screenshots.

- Escape AI-generated text before injecting it into HTML.
- Avoid logging full API secrets or sensitive learner input.
- Treat AI-generated content as untrusted output and validate structured quiz data.

7. Design Review Result

The proposed design is modular: one FastAPI entry point coordinates independent learning capabilities, while the browser remains lightweight. The architecture is suitable for incremental implementation and testing.